

Covid DB Use Cases

Joins

1. Which country has the highest number of confirmed cases on a specific date?

```
--1
select c.name, g.confirmed from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = '2021-09-30' order by g.confirmed desc limit 1;
```

	name character varying (100)	confirmed bigint
1	India	34500109

2. Show the total number of deaths in each country, including provinces/states, for a given date.

```
SELECT c.name AS country, g.report_date, SUM(g.deaths) AS total_deaths
FROM global_covid_stats g JOIN country c ON g.country_id = c.country_id
WHERE g.report_date = DATE '2021-09-30' GROUP BY c.name, g.report_date
ORDER BY total_deaths DESC;
```

	country character varying (100)	report_date date	total_deaths numeric
1	India	2021-09-30	621001
2	United States	2021-09-30	148951
3	Indonesia	2021-09-30	123085
4	Brazil	2021-09-30	95651
5	Nigeria	2021-09-30	92762
6	Russia	2021-09-30	65670
7	Mexico	2021-09-30	58019
8	Japan	2021-09-30	56914
9	Egypt	2021-09-30	46050
10	Turkey	2021-09-30	37952

3. List the continents along with the total number of confirmed cases, deaths, and recoveries.

```
--3
select c.continent,
       sum(g.confirmed) as total_confirmed, sum(g.deaths) as total_deaths, sum(g.recovered) as total_recovered
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats)
group by c.continent order by c.continent;
```

	continent character varying (50)	total_confirmed numeric	total_deaths numeric	total_recovered numeric
1	South America	27005824	486094	25385465
2	Oceania	2231238	40157	2097363
3	Africa	32180975	579248	30250108
4	Asia	163130042	2936323	153342237
5	North America	43546773	783829	40933962
6	Europe	41133247	740378	38665244

Aggregate Functions:

4. Calculate the average number of new deaths per day across all countries.

```
select avg(g.new_deaths) as average_new_deaths_per_day from covid.global_covid_stats g;
```

	average_new_deaths_per_day numeric
1	11926.866666666667

5. Find the maximum number of active cases recorded in any country on a specific date.

```
select c.name, g.active_cases from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date= '2021-09-30' order by g.active_cases desc limit 1;
```

	name character varying (100)	active_cases bigint
1	India	1449006

Stored Procedure:

6. Create a stored procedure that returns the total number of recovered cases for a given country and date.

```
--1
create or replace procedure get_recovered_cases(a_country varchar, a_date date)
language plpgsql
as $$
declare
    total_recovered bigint;
begin
    select g.recovered into total_recovered from covid.country c
    inner join covid.global_covid_stats g on c.country_id = g.country_id
    where c.name = a_country and g.report_date = a_date;
    raise notice 'Total recovered cases: %', total_recovered;
end;
$$;
call get_recovered_cases('India', '2021-09-30');
```

```
NOTICE: Total recovered cases: 32430102
CALL
```

```
Query returned successfully in 131 msec.
```

7. Design a stored procedure to update the number of deaths for a specific country and date.

```
create or replace procedure update_deaths(p_country varchar, p_date date, p_deaths bigint)
language plpgsql as
$$
begin
    update covid.global_covid_stats g
    set deaths = p_deaths from covid.country c
    where g.country_id = c.country_id and c.name = p_country and g.report_date = p_date;
end;
$$;
call update_deaths('india', '2021-09-30', 500);
```

```
NOTICE: Deaths Updated Successfully
CALL
```

```
Query returned successfully in 117 msec.
```

Views:

8. Create a view that displays the total number of cases (confirmed, deaths, and recovered) for each country on a specific date.

```
--1
create or replace view covid_cases_on_date as
select c.name as country, g.report_date, g.confirmed, g.deaths, g.recovered
from covid.country c inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = '2021-09-30';
```

CREATE VIEW

Query returned successfully in 112 msec.

	country character varying (100) 🔒	report_date date 🔒	confirmed bigint 🔒	deaths bigint 🔒	recovered bigint 🔒
1	India	2021-09-30	34500109	621001	32430102
2	United States	2021-09-30	8275066	148951	7778562
3	Brazil	2021-09-30	5313985	95651	4995145
4	United Kingdom	2021-09-30	1697150	30548	1595321
5	Russia	2021-09-30	3648361	65670	3429459
6	France	2021-09-30	1631837	29373	1533926

9. Implement a view to show the latest data (confirmed, deaths, recovered) for each country.

```
--2
create or replace view latest_country_data as
select distinct on (c.name) c.name as country, g.report_date, g.confirmed, g.deaths, g.recovered
from covid.country c inner join covid.global_covid_stats g on c.country_id = g.country_id
order by c.name, g.report_date desc;
```

CREATE VIEW

Query returned successfully in 112 msec.

	country character varying (100)	report_date date	confirmed bigint	deaths bigint	recovered bigint
1	Argentina	2021-09-30	1129894	20338	1062100
2	Australia	2021-09-30	637497	11474	599247
3	Brazil	2021-09-30	5313985	95651	4995145
4	Canada	2021-09-30	943553	16983	886939
5	Colombia	2021-09-30	1272072	22897	1195747
6	Egypt	2021-09-30	2558360	46050	2404858

T-SQL:

10. Write a T-SQL query to calculate the total number of cases (confirmed + deaths + recovered) for each country.

```
select c.name as country, g.confirmed+g.deaths+g.recovered as total_cases
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats);
```

	country character varying (100)	total_cases bigint
1	India	67551212
2	United States	16202579
3	Brazil	10404781
4	United Kingdom	3323019
5	Russia	7143490
6	France	3195136

11. Use T-SQL to identify the country with the highest number of new cases reported on a specific date.

```
select c.name as country, g.new_confirmed from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = '2021-09-30' order by g.new_confirmed desc limit 1;
```

	country character varying (100)	new_confirmed bigint
1	India	7245022

CTE (Common Table Expressions):

12. Create a CTE to calculate the percentage increase in confirmed cases for each country over the past week.

```
with weekly_data as
(select c.name as country, min(g.confirmed) as starting_cases, max(g.confirmed) as ending_cases
from covid.country c join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date >= '2021-09-24' and g.report_date <= '2021-09-30' group by c.name)
select country,
((ending_cases - starting_cases) * 100.0 / nullif(starting_cases, 0)) as percentage_increase
from weekly_data;
```

	country character varying (100) 🔒	percentage_increase numeric 🔒
1	Turkey	20.0000227652622587
2	Nigeria	20.0000046570393245
3	Indonesia	20.0000070195043949
4	Italy	20.0000793890220860
5	Russia	20.0000328914909713
6	United Kingdom	20.0000565654451594

13. Use a CTE to find the country with the highest number of active cases at the moment.

```
with current_cases as
(select c.name as country, g.active_cases from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats))
select country, active_cases from current_cases order by active_cases desc limit 1;
```

	country character varying (100) 🔒	active_cases bigint 🔒
1	India	1449006

Indexes

14. Explain the importance of indexes in optimizing queries for this dataset

Indexes improve query performance by allowing the database to find rows faster instead of scanning the entire table. They are especially useful for columns frequently used in WHERE, JOIN, ORDER BY, and GROUP BY clauses.

15. Implement an index on the "Country/Region" column to speed up search operations.

```
create index idx_country_name on covid.country(name);
```

	indexname name
1	country_pkey
2	country_name_k...

User-Defined Functions (UDF)

16. Develop a UDF to calculate the mortality rate (deaths / confirmed cases * 100) for a given country.

```
CREATE OR REPLACE FUNCTION mortality_rate(p_country VARCHAR)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_rate NUMERIC;
BEGIN
    SELECT CASE
        WHEN g.confirmed = 0 OR g.confirmed IS NULL THEN 0
        ELSE ROUND((g.deaths::NUMERIC / g.confirmed) * 100, 2)
    END
    INTO v_rate
    FROM covid.global_covid_stats g
    JOIN covid.country c ON g.country_id = c.country_id
    WHERE LOWER(c.name) = LOWER(p_country)
    ORDER BY g.report_date DESC
    LIMIT 1;

    RETURN v_rate;
END;
$$;
SELECT mortality_rate('india');
```

	mortality_rate numeric
1	1.80

17. Create a UDF to determine the recovery rate (recovered / confirmed cases * 100) for a specific date.

```
create or replace function recovery_rate(p_country varchar, p_date date)
returns numeric
language sql
as $$
select case
  when g.confirmed = 0 then 0
  else (g.recovered::numeric / g.confirmed) * 100
end
  from covid.global_covid_stats g join covid.country c on g.country_id = c.country_id
 where c.name = p_country and g.report_date = p_date; $$;

select recovery_rate('india', '2021-09-30');
```

	calculate_recovery_rate numeric
1	94.00

Group By

18. Group the data by continent and calculate the total number of confirmed cases for each continent.

```
select c.continent, sum(g.confirmed) as total_confirmed from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats)
group by c.continent order by total_confirmed desc;
```

	continent character varying (50)	total_confirmed numeric
1	Asia	163130042
2	North America	43546773
3	Europe	41133247
4	Africa	32180975
5	South America	27005824
6	Oceania	2231238

19. Group the data by date and compute the total number of deaths and recoveries for each date.

```
select report_date, sum(deaths) as total_deaths, sum(recovered) as total_recovered
from covid.global_covid_stats group by report_date order by report_date;
```

	report_date date	total_deaths numeric	total_recovered numeric
1	2020-06-30	265043	13841630
2	2020-09-30	530095	27683271
3	2020-12-31	795147	41524912
4	2021-03-31	1060196	55366547
5	2021-06-30	1325244	69208186
6	2021-09-30	1590304	83049833

20. Group the data by country and calculate the average number of new cases reported daily for each country.

```
select c.name as country, avg(g.new_confirmed) as average_new_cases
from covid.country c inner join covid.global_covid_stats g on c.country_id = g.country_id
group by c.name order by average_new_cases desc;
```

	country character varying (100)	average_new_cases numeric
1	India	6210019.000000000000
2	United States	1489511.000000000000
3	Indonesia	1230855.666666666667
4	Brazil	956516.666666666667
5	Nigeria	927627.666666666667
6	Russia	656704.333333333333

Covid-Data-Global :

1. To find out the death percentage locally and globally

```
--1

--globally
select (sum(g.deaths)::numeric / nullif(sum(g.confirmed), 0)) * 100 as global_death_percentage
from covid.global_covid_stats g
where g.report_date = (select max(report_date) from covid.global_covid_stats);

--locally
select c.name as country, (max(g.deaths)::numeric / nullif(max(g.confirmed), 0)) * 100 as death_percentage
from covid.country c
join covid.global_covid_stats g on c.country_id = g.country_id
group by c.name order by death_percentage desc;
```

	global_death_percentage numeric
1	1.79998625254478174700

	country character varying (100)	death_percentage numeric
1	United States	1.79999772811479690900
2	India	1.79999721160301261700
3	France	1.79999595547839643300
4	Argentina	1.79999185764328335200
5	Indonesia	1.79999093314068694600
6	Japan	1.79998741267543583600

2. To find out the infected population percentage locally and globally

```
--2

--globally
select (sum(g.confirmed)::numeric / nullif(sum(c.population), 0)) * 100 as global_infected_percentage
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats);

--locally
select c.name as country,
       (g.confirmed::numeric / nullif(c.population, 0)) * 100 as infected_population_percentage
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats)
order by infected_population_percentage desc;
```

	global_infected_percentage numeric
1	2.49999972764852202200

	country character varying (100)	infected_population_percentage numeric
1	India	2.49999995471028883700
2	United States	2.49999991691909440300
3	Egypt	2.49999990228115268100
4	Indonesia	2.49999986290032032500
5	Brazil	2.49999980005590625000
6	Nigeria	2.49999964829657247400

3. To find out the countries with the highest infection rates

```
--3
select c.name as country,
       (max(g.confirmed)::numeric/nullif(c.population,0))*100 as infection_rate
from covid.global_covid_stats g
inner join covid.country c on c.country_id=g.country_id
group by c.name, c.population order by infection_rate desc;
```

	country character varying (100)	infection_rate numeric
1	India	2.49999995471028883700
2	United States	2.49999991691909440300
3	Egypt	2.49999990228115268100
4	Indonesia	2.49999986290032032500
5	Brazil	2.49999980005590625000
6	Nigeria	2.49999964829657247400

4. To find out the countries and continents with the highest death counts

```
--4
--country
select c.name as country, max(g.deaths) as highest_deaths
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
group by c.name
order by highest_deaths desc;

--continents
select c.continent, sum(g.deaths) as total_deaths
from covid.country c
join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats)
group by c.continent
order by total_deaths desc;
```

	country character varying (100) 🔒	highest_deaths bigint 🔒
1	India	621001
2	United States	148951
3	Indonesia	123085
4	Brazil	95651
5	Nigeria	92762
6	Russia	65670

	continent character varying (50) 🔒	total_deaths numeric 🔒
1	Asia	838952
2	North America	223953
3	Europe	211539
4	Africa	165500
5	South America	138886
6	Oceania	11474

5. Average number of deaths by day (Continents and Countries) [Hint: order by , group by clause]

```
--5
-- country
select * from global_covid_stats;
select c.name as country, avg(g.new_deaths) as average_new_deaths
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
group by c.name
order by average_new_deaths desc;

-- continent
select c.continent, avg(g.new_deaths) as average_new_deaths
from covid.country c
join covid.global_covid_stats g on c.country_id = g.country_id
group by c.continent
order by average_new_deaths desc;
```

	country character varying (100) 🔒	average_new_deaths numeric 🔒
1	India	93150.0000000000000
2	United States	22342.0000000000000
3	Indonesia	18462.0000000000000
4	Brazil	14347.3333333333333
5	Nigeria	13914.0000000000000
6	Russia	9850.0000000000000

	continent character varying (50) 🔒	average_new_deaths numeric 🔒
1	Asia	31460.3333333333333
2	North America	11197.1111111111111
3	Africa	8274.6666666666667
4	South America	6944.0000000000000
5	Europe	5287.9444444444444
6	Oceania	1721.0000000000000

6. Average of cases divided by the number of population of each country (TOP 10) [Hint: Limit]

```
--6 |
select c.name as country,
       avg((g.confirmed::numeric / nullif(c.population, 0)) * 100) as average_infection_percentage
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
group by c.name order by average_infection_percentage desc limit 10;
```

	country character varying (100) 🔒	average_infection_percentage numeric 🔒
1	India	1.45833327672119438000
2	United States	1.45833318416534373900
3	Indonesia	1.45833313149213825700
4	Brazil	1.45833302067565732300
5	Colombia	1.45833301806691762100
6	Egypt	1.45833295060118133200

7. Considering the highest value of total cases, which countries have the highest rate of infection in relation to population? [Hint: Where clause]

```
select c.name as country, g.confirmed as highest_cases,
       (g.confirmed::numeric / nullif(c.population, 0)) * 100 as infection_rate
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.confirmed = (
    select max(confirmed) from covid.global_covid_stats
)
order by infection_rate desc;
```

	country character varying (100) 🔒	highest_cases bigint 🔒	infection_rate numeric 🔒
1	India	34500109	2.49999995471028883700

8. Countries with the highest number of deaths

```
--8
select c.name as country, max(g.deaths) as highest_deaths
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
group by c.name order by highest_deaths desc;
```

	country character varying (100) 🔒	highest_deaths bigint 🔒
1	India	621001
2	United States	148951
3	Indonesia	123085
4	Brazil	95651
5	Nigeria	92762
6	Russia	65670

9. Continents with the highest number of deaths

```
--9
select c.continent, sum(g.deaths) as total_deaths
from covid.country c
inner join covid.global_covid_stats g on c.country_id = g.country_id
where g.report_date = (select max(report_date) from covid.global_covid_stats)
group by c.continent order by total_deaths desc;
```

	continent character varying (50) 🔒	total_deaths numeric 🔒
1	Asia	838952
2	North America	223953
3	Europe	211539
4	Africa	165500
5	South America	138886
6	Oceania	11474

Queries on Vaccination:

1. Total vaccinated with at least 1 dose over time (All countries)

```
--1
select v.date, sum(v.first_dose) as total_vaccinated from covid.vaccination v
group by v.date order by total_vaccinated asc;
```

	date date	total_vaccinated numeric
1	2021-01-...	48276
2	2021-01-...	58604
3	2021-01-...	99449
4	2021-01-...	195525
5	2021-01-...	251280
6	2021-01-...	365965

2. Percentage of the population vaccinated with at least the first dose until 30/9/2021 (Top 3)

```
--2
select v.date, c.name, s.name,
       ((v.first_dose::numeric)/nullif(s.population,0)*100) as people_vaccinated
from covid.vaccination v
inner join covid.state s on s.state_id=v.state_id
inner join covid.country c on s.country_id=c.country_id
where v.date= (
    select max(v2.date)
    from covid.vaccination v2
    where v2.state_id = v.state_id
    and v2.date <= '2021-09-30'
) order by people_vaccinated desc limit 3;
```

	country character varying (100)	state character varying (100)	people_vaccinated numeric
1	India	Dadra and Nagar Haveli and Daman and ...	94.90778335747835069000
2	India	Lakshadweep	75.22941176470588235300
3	India	Sikkim	72.12608167173970048600

3. To find out the population vs the number of people vaccinated

```
--3
select c.name as country, s.name as state, s.population, v.first_dose, v.second_dose
from covid.state s
inner join covid.country c on s.country_id = c.country_id
inner join covid.vaccination v on s.state_id = v.state_id;
```

	country character varying (100)	state character varying (100)	population bigint	first_dose bigint	second_dose bigint
1	India	Andaman and Nicobar Islan...	417036	23	0
2	India	Andaman and Nicobar Islan...	417036	23	0
3	India	Andaman and Nicobar Islan...	417036	42	0
4	India	Andaman and Nicobar Islan...	417036	89	0
5	India	Andaman and Nicobar Islan...	417036	124	0
6	India	Andaman and Nicobar Islan...	417036	239	0

4. To find out the percentage of different vaccine taken by people in a country

```
--4
select c.name, s.name,
       ((v.covaxin::numeric) / nullif(v.total_doses, 0) * 100) as covaxin_percentage,
       ((v.covishield::numeric) / nullif(v.total_doses, 0) * 100) as covishield_percentage,
       ((v.sputnik_v::numeric) / nullif(v.total_doses, 0) * 100) as sputnik_v_percentage
from state s
inner join vaccination v on s.state_id=v.state_id
inner join country c on s.country_id=c.country_id
```

	name character varying (100)	name character varying (100)	covaxin_percentage numeric	covishield_percentage numeric	sputnik_v_percentage numeric
1	India	Rajasthan	9.16570509446361224000	90.69422557281115706700	0.008499572560130900518000
2	India	Jharkhand	14.01974579630005774100	85.88475916662206498700	0.01388092088057518800
3	India	Lakshadweep	0.001383738052113789023500	99.96033284250607138100	0.0000000000000000000000
4	India	Jammu and Kashmir	2.15481708151808196000	97.78683456653786091800	0.0000000000000000000000

5. To find out percentage of people who took both the doses

```
--5
select s.name,
       (SUM(v.second_dose)::numeric / NULLIF(SUM(v.first_dose), 0)) * 100 as both_dose_percentage
from covid.state s
inner join covid.vaccination v on s.state_id=v.state_id
group by s.name order by both_dose_percentage desc;
```

	name character varying (100)	both_dose_percenatge numeric
1	Tripura	33.87318014038797273500
2	West Bengal	32.62806555089689168100
3	Ladakh	30.62320033820628820500
4	Kerala	30.49380204670309180600
5	Delhi	30.14689189770322274700
6	Andhra Pradesh	29.10446839602566825000
7	Gujarat	28.16688950533906401500
8	Uttarakhand	27.55710276282222792200
9	Sikkim	26.29366071360727579300
10	Chandigarh	25.93496581501441566500
11	Maharashtra	25.60301028364236720300

Indian State Wise Analysis:

1. Total State-wise Confirmed Cases

```
--1
select s.name as state, sum(cs.confirmed) as total_confirmed from covid.state s
inner join covid.covid_case_stats cs on s.state_id = cs.state_id
group by s.name order by total_confirmed desc;
```

2. Maximum Active cases State-wise till date

```
--2
select s.name as state, max(cs.active_cases) as maximum_active_cases from covid.state s
inner join covid.covid_case_stats cs on s.state_id = cs.state_id
group by s.name order by maximum_active_cases desc;
```

3. Max Per Day Confirmed cases in States

```
--3
select s.name as state, max(cs.new_confirmed) as max_per_day_confirmed from covid.state s
inner join covid.covid_case_stats cs on s.state_id = cs.state_id
group by s.name order by max_per_day_confirmed desc;
```

	state character varying (100)	max_per_day_confirmed integer
1	Maharashtra	68631
2	Karnataka	50112
3	Kerala	43529
4	Uttar Pradesh	37944
5	Tamil Nadu	36184
6	Delhi	28395
7	Andhra Pradesh	24171
8	West Bengal	20846
9	Rajasthan	18298
10	Chhattisgarh	17397
11	Bihar	15853

4. Max Per Day Death cases in States

```
--4
select s.name as state, max(cs.new_deaths) as max_per_day_deaths from covid.state s
inner join covid.covid_case_stats cs on s.state_id = cs.state_id
group by s.name order by max_per_day_deaths desc;
```

	state character varying (100)	max_per_day_deaths integer
1	Bihar	3971
2	Maharashtra	3656
3	Madhya Pradesh	1481
4	Karnataka	624
5	Tamil Nadu	518
6	Delhi	448
7	Uttar Pradesh	372
8	Chhattisgarh	279
9	Punjab	231
10	Kerala	227
11	Uttarakhand	223

5. State-wise Mortality Rate

```
--5
select s.name as state,
       (sum(cs.deaths)::numeric / nullif(sum(cs.confirmed), 0)) * 100 as mortality_rate
from covid.state s
inner join covid.covid_case_stats cs on s.state_id = cs.state_id
group by s.name order by mortality_rate desc;
```

	state character varying (100)	mortality_rate numeric
1	Punjab	2.78681526605404286300
2	Maharashtra	2.11649722463328682200
3	Uttarakhand	1.85546352725065333500
4	Delhi	1.72098013525291662100
5	Sikkim	1.66781776949220832600
6	Himachal Pradesh	1.63651217720094526200
7	Goa	1.58568866414668557600
8	Puducherry	1.55564983383992268300
9	Gujarat	1.54751550065352772600
10	West Bengal	1.46213372951252892200
11	Jammu and Kashmir	1.44481564884352151000